

CONFIDENCE-AWARE NIDS

CA-xNIDS

Confidence-Aware Explainability for Deep Learning-based
Network Intrusion Detection Systems

LSTM DL-NIDS

xNIDS Explainer

Bootstrap Confidence

Confidence Gating

KDD Cup 99

Based on: xNIDS — Feng Wei et al., USENIX Security 2023

Why Can't DL-NIDS Explain Themselves?

DL-NIDS detect attacks accurately — but operators need to know *why*, not just *that*

SEMANTIC GAP

The Operator Problem

- DL-NIDS output: an anomaly score or binary label
- Operator needs: *which* features caused the alert
- Without explanation → no active intrusion response
- High false-positive rates overwhelm SOC teams

METHODS GAP

Existing Methods Fall Short

- LIME / SHAP: assume feature independence (wrong for NIDS)
- LEMNA: assumes adjacent features are similar (false)
- Integrated Gradients: cannot handle LSTM history inputs
- None handle structured network feature group dependencies

ROOT CAUSE DL-NIDS operates on **temporal history sequences** AND **structured feature groups** — no existing explainer handles both simultaneously.

CH1

Incorporate LSTM history inputs into explanation

CH2

Capture inter-feature group dependencies

CH3

Balance precision vs. generalisation in rules

CH4

Generate iptables + OpenFlow rules uniformly

xNIDS Framework

History-aware explainer → sparse group-level importance → active defence rules

01

History Approximation

- Find minimal history subset I such that
- $|f(x, X_I) - f(x, X_k)| < \delta$
- Avoids degenerate explanations
- Focuses explainer on relevant context

02

WRS + Group Lasso

- Weighted Random Sampling — Gaussian decay
- Recent history inputs have higher weight
- Sparse Group Lasso ($\alpha=0.01$) finds
- group-level AND feature-level sparsity

03

Defence Rules

- Scope: per-flow | per-host | multi-host
- Action: drop_flow | drop_host | log
- Output: iptables rule string
- Output: OpenFlow flow entry

PAPER EVALUATION – xNIDS VS BASELINES (TABLE 8)

METRIC	LIME	SHAP	LEMNA	IG	LRP	xNIDS
Fidelity						
Sparsity						
Completeness						
Stability						
Rule Gen.						

Three Gaps in the xNIDS Paper

We identified one critical unaddressed weakness — our project directly solves it

GAP 1

CRITICAL

No Explanation Confidence Score

xNIDS uses stochastic sampling — running the same explanation twice with different random seeds can produce different top features. The paper measures stability as a *metric* but **never uses it to gate rule deployment**. Unreliable explanations auto-deploy rules that may block legitimate traffic.

GAP 2

KNOWN

Adversarial Robustness

Paper reports evasion rates of 33%, 49%, 53% under adversarial traffic manipulation. An attacker can craft packets to produce misleading explanations and bypass rule generation entirely. **No mitigation strategy is proposed.**

GAP 3

MINOR

Static Security Constraints

Block lists, IP whitelists, port allowlists, and block strategies (passive / assertive / aggressive) must all be manually configured. There is no mechanism to auto-adapt to changing network topologies or evolving attack patterns.

WE ADDRESS GAP 1

CA-xNIDS adds a bootstrap confidence score that quantifies explanation reliability and gates rule deployment accordingly — only reliable explanations auto-deploy defence rules.

CA-xNIDS — Bootstrap Confidence Scoring

If two random seeds produce different top features, the explanation is unreliable — so don't auto-deploy the rule

ALGORITHM

```
# Input: anomaly x, history X_h, predict_fn f # Params: N=10 bootstrap runs, top_k=5
for i in range(N): rng_i = RandomState(seed=base + i) Z = WRS_sample(x, X_h, rng_i) y
= f(Z) # LSTM + real history beta_i = SparseGroupLasso(Z, y, α=0.01) CV = std(betas)
/ (mean(betas) + ε) conf = 1 - mean(CV[top_k_features]) # Confidence tier if conf >=
0.75: tier = "HIGH" elif conf >= 0.50: tier = "MEDIUM" else: tier = "LOW"
```

CONFIDENCE-GATED DEPLOYMENT

HIGH

conf ≥ 0.75

Auto-deploy defence rule immediately

Explanation is stable across bootstrap seeds. iptables + OpenFlow rule generated.

MEDIUM

0.50 ≤ conf < 0.75

Generate rule — require operator review

Some instability detected. Operator must approve before deployment.

LOW

conf < 0.50

Flag for manual investigation only

Explanation too unstable to trust. No defence rule generated automatically.

System Implementation

KDD Cup 99 · PyTorch LSTM · WRS Explainer · CA-xNIDS Confidence Module



SRC/EXPLAINER.PY – WRS SAMPLING

```
# Gaussian decay weights – recent history highest w = exp(-pos² / 2σ²) # σ=1.5 Z =
t·x_current + (1-t)·x_history + N(0, σ) beta = SparseGroupLasso(Z, predict_fn(Z))
```

SRC/CONFIDENCE.PY – BOOTSTRAP [OUR CONTRIBUTION]

```
for i in range(N=10): beta_i = explainer.explain(pfn, x, hist, seed=i) CV =
std(betas) / (mean(betas) + ε) conf = 1 - mean(CV[top_k_features])
```

basic [0–8]	content [9–21]	time_traffic [22–30]	host_traffic [31–40]
duration, protocol_type, service, flag, src/dst_bytes, land...	hot, num_failed_logins, logged_in, root_shell, su_attempted...	count, srv_count, serror_rate, same_srv_rate...	dst_host_count, dst_host_srv_count, dst_host_serror_rate...

Interactive SOC Dashboard

FastAPI + React + WebSocket real-time stream · live xNIDS / CA-xNIDS explanations

Dashboard
/

· KPI summary cards

· Live traffic chart

· Attack distribution

· Confidence gauge

Threat Log
/threats

· Attacks only view

· Filter by type

· Inline expansion

· Live count badge

Live Detection
/detection

· Full packet stream

· Live / Lock toggle

· xNIDS Analysis tab

· Packet Inspector tab

Control Center
/control

· Start / Stop engine

· Mode selector

· Inject test traffic

· Injection log

Attack Studio
/attacks

· DoS/Probe/R2L/U2R

· Burst size slider

· Multi-vector attack

· Flash feedback

STACK

- React 18 + TypeScript
- Zustand
- Framer Motion
- Tailwind CSS
- FastAPI
- Python 3.11
- WebSocket
- PyTorch 2.x
- KDD Cup 99

DETECTION MODE TOGGLE (LIVE, NO RESTART)

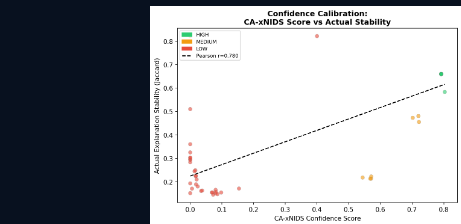
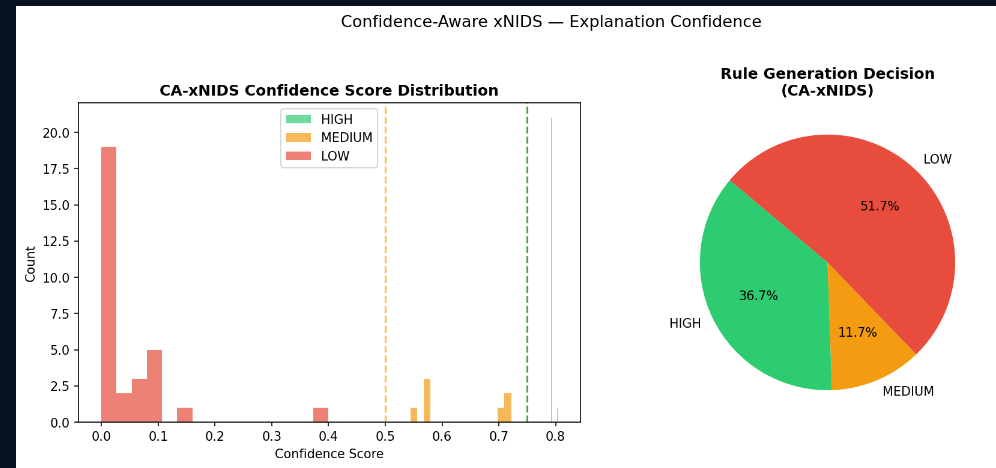
- xNIDS Mode (Paper)

Single LIME run · no confidence · all rules auto-deploy
- CA-xNIDS Mode (Ours)

10 bootstrap runs · confidence score · gated deployment

CA-xNIDS Confidence Scores Work

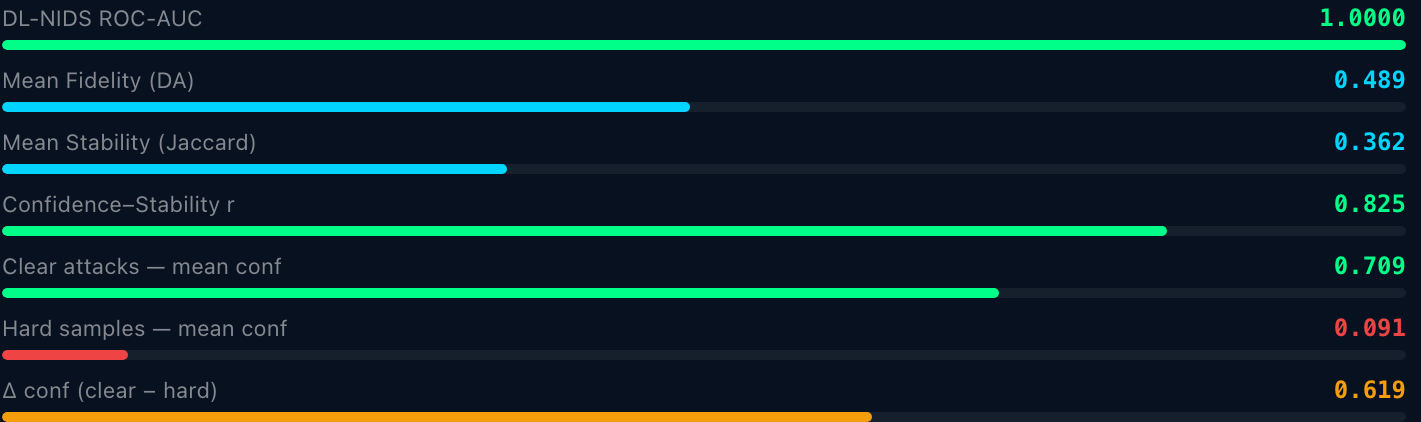
30 clear attacks vs 30 hard/borderline samples · 10 bootstrap runs each · KDD Cup 99



Key Insight — Pearson $r = 0.825$

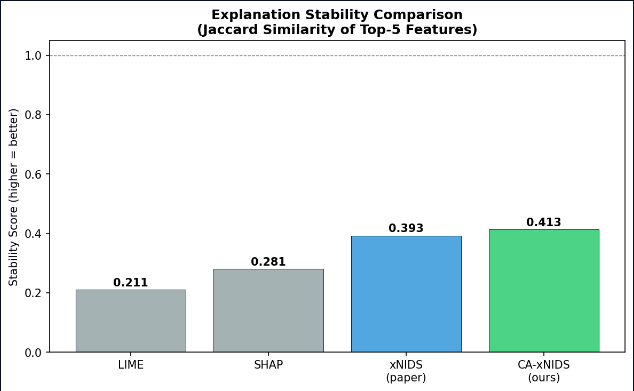
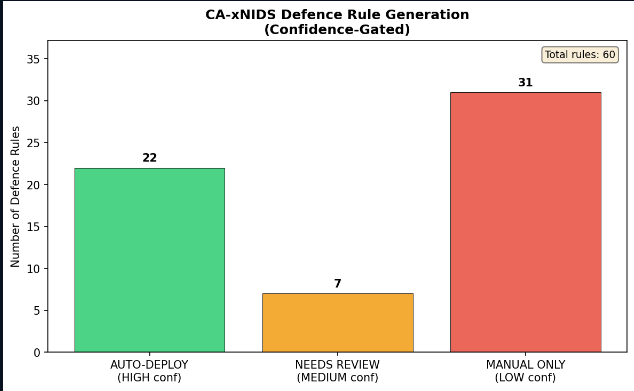
- Confidence score reliably predicts actual explanation stability
- HIGH conf (≥ 0.75) $\rightarrow$ mean Jaccard stability = 0.655
- MEDIUM conf $\rightarrow$ 0.519 | LOW conf (< 0.50) $\rightarrow$ 0.211

RESULTS



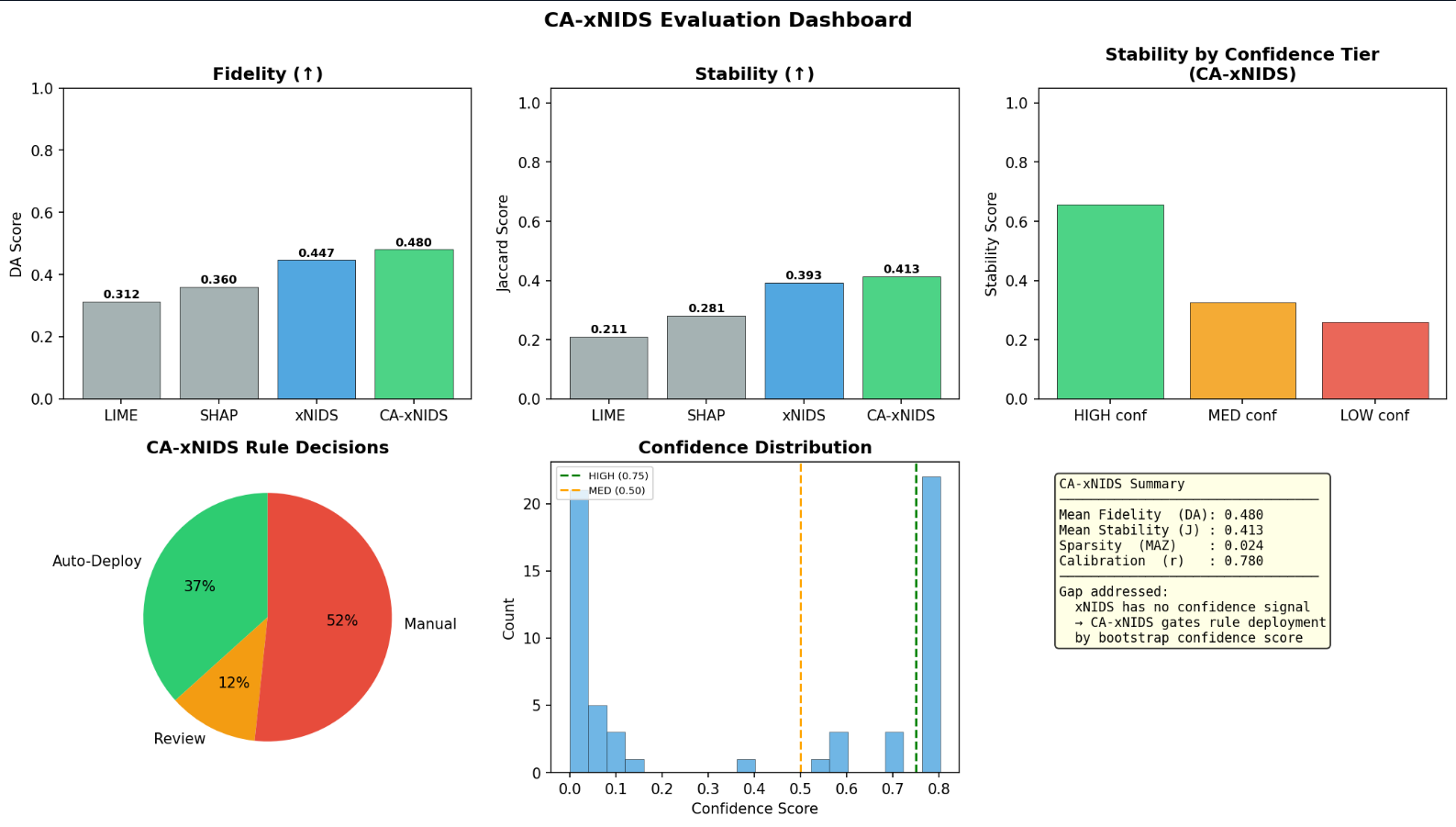
Confidence-Gated Defence Rules

Only HIGH-confidence explanations auto-deploy — MEDIUM needs review — LOW is blocked entirely



AUTO-DEPLOYED conf = 0.77	<pre>iptables -A INPUT -p tcp -m comment --comment "src_bytes=2.1; srv_count=1.8" -j DROP</pre>
NEEDS REVIEW conf = 0.62	<pre>OpenFlow: <nw_src=dst_bytes=1.4; error_rate=0.9, actions=drop, priority=7> *** REVIEW ***</pre>
BLOCKED conf = 0.18	<pre>No rule generated – explanation unstable across bootstrap seeds, manual investigation required</pre>

Full Evaluation Dashboard



CA-xNIDS vs xNIDS — What We Add

A drop-in enhancement: wraps the xNIDS explainer with a confidence layer — no changes to LSTM or rule engine

ASPECT	xNIDS (PAPER)	CA-xNIDS (OURS)
Explanation method	WRS + Sparse Group Lasso	Same + Bootstrap wrapper (N=10)
Confidence signal	None	Bootstrap score $\in [0, 1]$
Rule deployment	All anomalies $\rightarrow$ auto-deploy	Gated: HIGH / MEDIUM / LOW tier
False-rule prevention	No mechanism	LOW conf rules blocked
Operator guidance	Detection + explanation	Detection + explanation + tier badge
Calibration metric	Stability measured, not acted on	Pearson $r = 0.825$
Novel contribution	Not applicable	Confidence Calibration Score
Runtime mode switch	Not applicable	Live toggle in dashboard header

CA-xNIDS is fully backwards-compatible — same LSTM model, same xNIDS explainer, same rule format. It only adds the **confidence estimation layer** between explanation and deployment.

Team Contributions

CSL6010 Major Project — IIT Jodhpur M.Tech AI

M1

Paper Study & Analysis

Thorough reading of xNIDS. Identified Ch1–Ch4 challenges. Documented semantic gap and why existing methods fail for DL-NIDS.

M2

Architecture Design

Designed CA-xNIDS pipeline. Formulated bootstrap confidence metric and HIGH/MEDIUM/LOW tier thresholds.

M3

LSTM DL-NIDS

Implemented 2-layer LSTM in PyTorch (src/model.py). Training pipeline, evaluation loop, make_explain_predict_fn API.

M4

xNIDS Explainer

WRS sampling + Sparse Group Lasso (src/explainer.py). KDD99 feature group definitions from domain knowledge.

M5

CA-xNIDS Confidence Module

Core contribution: bootstrap confidence scoring (src/confidence.py). Confidence calibration evaluation. Our novel metric.

M6

Defence Rules & Evaluation

Rule generation + Fidelity/Sparsity/Stability/Calibration metrics (src/defense_rules.py, src/evaluate.py).

M7

Dashboard, Slides & Submission

Full-stack SOC dashboard (FastAPI + React). All result visualisations. This PDF presentation. ZIP submission.

Conclusion

- xNIDS explains DL-NIDS using history-aware WRS + sparse group lasso — outperforms LIME, SHAP, LEMNA
- ! **GAP: no confidence signal means unreliable explanations can auto-deploy bad defence rules**
- ★ CA-xNIDS adds N=10 bootstrap confidence scoring — Pearson $r = 0.825$ validates it
- ★ Clear attacks: conf = 0.709 | Hard samples: conf = 0.091 ($\Delta = 0.619$)
- ★ Confidence-gated deployment prevents false-positive defence rules in production